codewiki 使用手册

把三端代码与旧文档 compound 成可被验证的 LLM Wiki

codewiki 由 Go CLI `kb` 和一组 Agent Skills 组成，Claude Code、Codex、Gemini CLI 等 agent 都能直接使用。它把移动端 monorepo（Android Kotlin/Java、iOS Swift、HarmonyOS ArkTS，另支持 JS/TS）和遗留文档整理成适合 agent 使用的知识库。每次 merge 到 master 都会做一次增量更新。每个页面都通过“符号 ID + 内容 hash”锚定到代码，CI 中的 `kb verify` 负责证明这些知识与当前代码一致。

1. 项目目标
2. 架构设计：三层结构与 compound 流程
3. 核心能力：命令一览
4. 使用方法：安装、配置、CI、skills、多 Agent 支持
5. Demo 操作流程与评测（含真实 agent 使用 skills）
6. 真实项目验证与性能（23 个公开仓库）
7. 最佳实践与引用
8. 局限与路线图

版本 0.1 · 2026-10-20 · 模块 `github.com/luckylucky0001/codewiki` · Go 1.24 + tree-sitter（cgo）· 文中所有数字均来自本仓库 `demo/sample-output/` 中的实际运行结果

1 项目目标

移动团队的“知识”通常散落在三端代码、过时的 wiki 和口口相传里。每次让 LLM 从零读仓库，成本高，结果也不稳定。LLM 一次性生成的文档则会漂移：代码改了，文档没改，也没人知道哪一页已经过时。codewiki 想解决三个问题：

1. 知识复利：沿用 Karpathy 的 LLM Wiki 模式（raw sources / wiki pages / schema）。每次合并只重写受影响的页面，知识持续累积，不必每次从头生成。
2. 可证明一致：页面 frontmatter 的 `sources` 记录“符号 ID + 归一化内容 hash”。CI 中 `kb verify` 检查所有锚点与 HEAD 是否一致，不一致就不发布。
3. 三端对齐：用同一套 `route_id`、testID、API 描述 Android / iOS / 鸿蒙的同一个功能，并输出机器可读的 facts，供 e2e 测试（兄弟项目 e2epilot）、coding agent 和 CI 使用。

> 设计原则：确定性的事情交给 CLI（tree-sitter 抽取、diff、plan、verify），理解交给 LLM（写解释和语义核对），一致性靠硬校验（锚点 hash、写集、staging 验证后再原子发布）。

目标用户与场景

角色	场景	使用方式
Coding agent	改代码前了解一个文件影响哪些功能与测试	`kb context <file>`、`kb mcp` 工具
e2e 测试（e2epilot）	根据变更只重跑受影响路由；建立需求 → 流程的追踪	`routes/testids/apis/requirements.json`、`changed_route_ids`
Tech lead / 新人	理解三端实现差异与共享组件	`pages/features/*`、`parity.md`、`kb search`
CI	merge 后自动更新知识并设置门禁	`.github/workflows/compound.yml`
文档负责人	找出与代码冲突的 spec 和旧文档	`kb discover-docs` + `kb ingest` → `_conflicts.md`

交付物一览

24
kb 子命令（全部支持 --json）

7
tree-sitter 语言：Kotlin/Java/Swift/ArkTS/JS/TS/TSX

4
Agent Skills：compound / ingest / query / lint

120
Go 测试函数（单元、集成、评测、安全、真实 agent CLI），vet / gofmt 均无告警

2 架构设计

2.1 三层结构（Karpathy LLM Wiki）



图 1　三层结构：facts 由 CLI 确定性生成，pages 由 LLM 在受限写集内书写，全部经过 verify 门禁

2.2 Compound 流程（每次 merge 到 master）

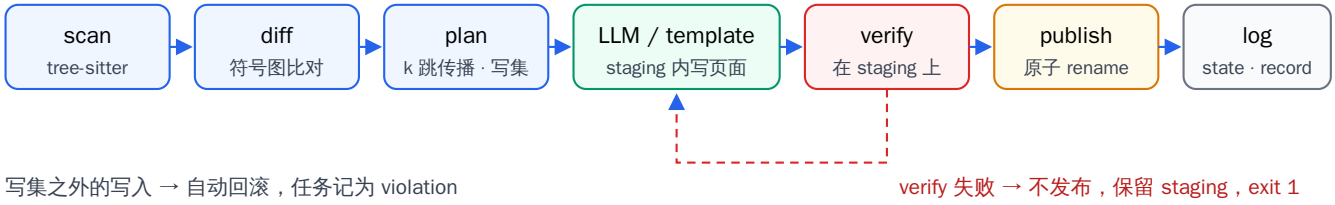


图 2　compound 流水线：基线是上次成功运行保存的 `.kb/graph.json`（不依赖 commit 链），因此 squash 合并或积压多次 merge 都能一次追平

2.3 符号模型与增量判定

字段	含义
`id`	`<platform>://<相对平台根路径>#<限定名>`，例如 `android://app/src/main/java/com/minishop/ui/cart/CartViewModel.kt#CartViewModel.addItem`
`hash`	去掉注释、归一化空白后 token 序列的 sha256。只改注释或格式视为 no-op。容器（class/struct）只计入自身头部和成员签名
`sig_hash`	声明头（含注解、装饰器）的 hash，用来区分 body 变化和 signature 变化
`refs`	函数体内引用的已知符号，按同文件 → 父类型被引用 → 全局唯一（结合调用点信息）的顺序解析
`norm_hash` / `simhash`	去掉自身名字后的 body 指纹，用于识别 rename（完全一致记 1.0，同目录 simhash ≥ 0.8 记模糊 rename）

plan 规则：页面被判为受影响，要么是它自己的锚点发生了 body / signature 变化、被删除或改名，要么它锚定了“签名变化符号”k 跳以内的调用方（沿反向引用边传播，参考 RepoAgent）。facts 中出现新 `route_id` 时创建功能页。模块变化超过 50%，或结构变化（增、删、改名、签名变化）超过 30% 时，改为全量重建。

锚点精度：功能页只锚定路由所属的 screen、它的 ViewModel / store，以及 2 跳以内的接口。被 3 个及以上路由引用的符号归入 `concepts/shared-components.md`，路由器和深链处理代码归入 `concepts/navigation.md`。这样修改 `Back Button` 时只会重新检查共享组件页（有单测覆盖）。

3 核心能力

所有命令都支持 `--json`、`--root`、`--config`、`--wiki-dir`、`--quiet`：数据写 stdout，日志写 stderr。退出码：0 成功，1 存在 stale 或 verify 失败，2 用法错误，3 内部错误。写文件的命令支持 `--dry-run`；全部非交互、可幂等重跑。

命令	作用
`kb init`	探测平台目录，生成 `codewiki.yaml`、wiki 骨架、`AGENTS.md` schema 和 `.gitignore` 条目
`kb scan`	tree-sitter 并行抽取符号和 facts（按内容 hash 缓存），报告 `parse_failures`
`kb diff [--base sha]`	按符号 ID 比较新旧图，分为 unchanged / body / signature / added / deleted / renamed，并输出 `changed_route_ids` 和 `changed_files`
`kb plan`	为每个受影响页面生成任务，带 reasons 和显式写集，写入 `.kb/plan.json`；超过阈值时改为全量
`kb compound`	scan → diff → plan → LLM → verify → 原子发布。支持 `--llm template|claude`、`--fallback`、`--max-llm-tasks`
`kb verify / status`	CI 门禁：锚点 hash、符号是否存在、死链、孤儿页、index 覆盖、facts / parity 是否最新；`status` 列出 stale 页面
`kb discover-docs`	按 spec-kit、OpenSpec、Kiro、ADR、RFC、AGENTS.md、README、docs/ 的约定发现文档并分类
`kb ingest`	把文档复制到 `raw/docs`，抽取断言（EARS、MUST/SHALL、FR-001、Kiro 编号），逐条判为 verified / conflict / unverifiable
`kb search / get / context`	检索 wiki；按 id 取页面；查询某个文件对应的页面、符号、路由、testID、接口，以及锚定在该文件中的文档需求（供测试生成使用）
`kb conflicts`	哪些文档过时：与代码矛盾的断言，附 `文件:行号`、代码证据和修改建议（MCP：`kb_conflicts`）
`kb routes / lint-tid / parity`	按模块汇总路由并列出 dangling buildFunction；testID 覆盖矩阵（`--pages`：哪些页面设置了 testID；`--top`：最常用的 testID）；功能 × 平台一致性和跨平台常量差异
`kb bench / eval`	冷 / 热扫描基准；用人工标注的 golden set 评测断言核对与 plan 影响面（precision / recall）
`kb mcp / anchor / install / agents`	stdio MCP server（7 个工具）；打印符号当前 hash；按 agent 安装 skill、指引块和 MCP 注册，可重复执行（`install-skills` 为别名）；列出 agent 支持矩阵
`kb hook install`	安装 git 钩子：`post-receive`（服务端 push 到 master 时 compound；有锁串行，能追平进度）/ `post-merge`（客户端）。配合全局参数 `--wiki-dir`

三端 facts 来源

	路由 / 页面	深链	testID	接口
Android	Compose `composable(Routes.X)`，解析 `const val`	Manifest `<data scheme>`	`Modifier.testTag`	Retrofit、Ktor、OkHttp
iOS	`enum Route: String` + `XxxView`	Info.plist `CFBundleURLSchemes`	`.accessibilityIdentifier`	URLSession、Alamofire、Moya
鸿蒙	`router_map.json`（name / pageSourceFile / buildFunction）、`main_pages.json`	`module.json5` skills.uris	`.id('x')`（含 ArkUI 链式调用）	http、@ohos/axios、rcp

动态 testID（`product_item_${id}`）按前缀记录；拼接、插值和常量引用静态求值。JSON 与 e2epilot 共用 schema。

4 使用方法

4.1 安装与初始化

```
go install \
  github.com/luckylucky0001/codewiki/cmd/kb@lates
t
cd your-monorepo
kb init           # 配置 + wiki 骨架
kb discover-docs --json > docs.json
kb ingest --manifest docs.json
kb compound       # 首次全量；--llm claude
kb verify         # CI 门禁
```

4.2 配置 codewiki.yaml

```
version: 1
wiki_dir: wiki
platforms:
  - {name: android, root: android}
  - {name: ios,     root: ios}
  - {name: harmony, root: harmony}
llm:
  cmd: claude -p --permission-mode ...
plan:
  hops: 1
  rebuild_module_ratio: 0.5
```

指定 `--config <file> --root <repo>` 时，wiki 会写到被分析仓库之外，用于只读分析第三方仓库（axios 和 5 个开源项目都是这样跑的）。

4.3 页面 frontmatter 契约

```
---
id: features/cart          # = 路径去掉 pages/ 与 .md（verify 强制检查）
title: 购物车（cart）
kind: feature              # feature | concept | overview | conflicts | parity
platforms: [android, harmony, ios]
routes: [cart]
sources:
  - sym: android://app/src/main/java/com/minishop/ui/cart/CartScreen.kt#CartScreen
    hash: 298d51bd3abcc95697e472891f857dc8…
source_commit: 516a1c3
status: current            # current | stale | outdated-doc
---
```

4.4 CI、Skills、MCP 与 e2epilot

接入点	做法
CI	`.github/workflows/compound.yml` ：push 到 master/main 时触发， `concurrency` 串行且不取消； `actions/cache` 缓存 kb 和 `wiki/.kb/cache` ；有 `ANTHROPIC_API_KEY` 时用 `--agent claude` ，否则有 `OPENAI_API_KEY` 时用 `--agent codex` ，都没有时用 template。 `--max-llm-tasks` 按价值排序分配预算（feature > flow > concept，同类再按变化锚点数排序）。verify 失败则 job 失败；成功时以 `[skip ci]` 提交 wiki
Skills	`kb install --agent …` 安装 `kb-compound / kb-ingest / kb-query / kb-lint` ：Claude 装到 `.claude/skills` ，Codex 和 Gemini 装到 `.agents/skills` 。skill 只负责调用 CLI，不重新实现逻辑（实测见 5.4 和 4.5）
自建 git	`kb hook install --mode post-receive --repo app.git` 。适用于 GitLab server hooks、Gerrit 和自建 git：push 到 master 后 compound，wiki 放在服务端，不回写 master（实测见 5.6）
MCP	`kb install --agent … --mcp` 会写入 `.mcp.json` 、 `.codex/config.toml` 、 `.gemini/settings.json` 、 `.cursor/mcp.json` ，命令为 `kb mcp --root <repo>` 。提供 7 个只读工具： `kb_search / get / context / anchor / status / parity / conflicts` 。Claude、Codex 和 Gemini 的真实 CLI 都已完成握手
e2epilot	`kb diff --json` 和 `plan.json` 输出 `changed_route_ids` 、 `changed_routes` （附原因）和 `changed_files` （含 git 资源文件），供 `e2e graph invalidate --changed-routes` 和 `e2e impact` 使用

4.5 多 Agent 支持（`kb install --agent` / `kb compound --agent`，详见 docs/agents.md）

Agent	后端：`--agent` 实际执行的命令（prompt 从 stdin 读入，cwd 为 staging）	消费者：`kb install` 写入的文件	验证程度
Claude Code	`claude -p --permission-mode acceptEdits --allowedTools … --output-format json`	`CLAUDE.md` 块、`.claude/skills`、`.mcp.json`	实跑
Codex CLI	`codex exec --sandbox workspace-write -c approval_policy="never" --skip-git-repo-check --ephemeral --json -`	`AGENTS.md` 块、`.agents/skills`、`.codex/config.toml`（项目需被信任）	真实二进制 + mock 模型；CLI 核对
Gemini CLI	`gemini --skip-trust --approval-mode auto_edit -o json -p …`	`GEMINI.md` 块、`.agents/skills`、`.gemini/settings.json`（`trust:true`）	真实二进制 + mock 模型
Cursor / Copilot	无适配器，用 `--llm-cmd`	`.cursor/rules/codewiki.mdc`、`.cursor/mcp.json` / `.github/copilot-instructions.md`	仅文档

守卫与 agent 无关：写集合回滚和 `<untrusted-data>` 包裹由 kb 执行，agent 沙箱只是外层防线。Codex 的 workspace-write 实测仍可写 `/tmp` 和整个 staging；Gemini 的 auto_edit 可写工作区内任意文件。

验证：

- Codex 0.163.0 和 Gemini 0.65.0 的真实二进制通过本地 mock 模型 API 跑通了 `kb compound`：页面写入成功，越界文件被回滚，用量从各自的 JSON 事件中解析。
- `codex debug prompt-input` 确认模型能看到 AGENTS.md 块和 4 个 skill。
- `kb install` 会提示 Codex 的两个坑：`AGENTS.override.md` 会取代 AGENTS.md，超过 32 KiB 的部分会被截断。
- 真实 Claude（haiku）：`kb compound --agent claude` 完成 2 个任务，花费 $0.032；`kb ingest --agent claude` 用 1 个任务、花费 $0.0075，找出 2 处文档冲突。安装后的问答首选 kb 命令，只开放 MCP 时也能完成。
- A/B 对比 `--agent claude` 与 `--agent all`（18 次运行）：两组都没有读取其他 agent 的文件，首个动作相同，花费相同。
- `demo/codex-mock-demo.sh` 区分了真实环节和 mock 环节。真实的有：codex 二进制、provider 配置、沙箱、`exec_command` 工具、`--json` 事件流、kb 的 prompt、守卫和 verify。mock 的只有模型输出。
- 这次运行发现并修复了一个 bug：agent 的 stdout 和 stderr 由两个 goroutine 并发写入同一个日志，事件行会丢失，或者被 stderr 从中间截断拼接。修复方法是每个流按行缓冲，只把整行写入日志。用 `-race -count=50` 验证过。

5 Demo 操作流程与评测

`demo/run.sh` 用 `examples/minishop` 建一个临时 git 仓库。这是同一个电商 App 的三端实现，包含 6 条路由、11 类 testID、5 个接口。脚本依次执行 init → discover-docs → ingest → 首次 compound，然后把 3 个变更集以 `--no-ff` 合并到 master，每次合并后运行 status / diff / plan / compound / verify / parity，最后运行 `kb eval`。`DEMO_LLM=template` 约 1 分钟，结果可复现；`DEMO_LLM=claude` 使用真实的 `claude -p`。

5.1 文档发现与断言核对

共发现 9 份文档：spec-kit 的 `specs/001-cart/spec.md` 和 `plan.md`，Kiro 的 requirements 和 steering，ADR，3 份旧文档，以及 README（不参与 ingest）。从中抽取 33 条断言。三次合并后，template 模式结果为 verified 22 / conflict 10 / unverifiable 1（第 4 轮为 24 / 6 / 3）。第 5 轮加入的值规则和行为规则，让 template 也能识别短信对话框、分页参数、`share_button`、共享常量这几类冲突。

断言（来源）	结论与证据（以代码为准）
登录使用短信验证码，`POST /api/sms/send`（旧文档）	conflict：代码中不存在该接口，只有 **`POST /api/login`**
购物车最多 10 件（`MAX_CART_ITEMS`）	conflict：android 与 harmony 为 50，ios 为 99
LOGIN-R2.2：用户名为空时弹出短信验证对话框（Kiro）	conflict：代码中没有任何短信 / 验证码相关实现（第 4 轮 template 误判为 verified，只有 claude 能识别）

5.2 三次合并的增量结果（template 模式实测）

步骤	diff	changed_route_ids	任务
首次 compound	无基线，执行全量	—	10 个任务：6 个功能页、总览、2 个 concept 页、_conflicts
01 上限 99→50（iOS 漏改），另有一处只改注释	body 2，unchanged 212，只改注释的部分判为 no-op	`cart, product_detail`	3 个任务：cart、product_detail、_conflicts
02 ArkTS 改名并修改签名	renamed 1（`loadProducts→fetchProducts`，相似度 1.00），signature 1	`product_detail, products`	4 个任务
03 三端新增 profile 路由	added 10	`profile`（route_added）	4 个任务：create profile、overview、navigation、shared

每一步 verify 都通过，最终得到 24 个页面、377 个锚点。`kb parity` 报告 `max_cart_items` 三端分别为 `50 / 99 / 50`。在修改 `BackButton` 的单测中，只有 shared-components 页被重新检查。

5.3 评测：`kb eval`（开发集 33 条 / 盲测集 31 条）

开发集（`golden.yaml`）有 33 条断言，并标注了每个变更集应该重检的页面；第 5 轮的规则就是看着它写的。盲测集（`heldout/`）由另一个 agent 编写，它没有读过规则代码，写了一份新的旧文档，共 31 条断言。

数据集 · 检查器	准确率	conflict P / R	verified P / R
开发集 · template 第 4 轮	0.82	1.00 / 0.56	0.88 / 0.92
开发集 · template 第 5 轮	0.97	1.00 / 1.00	1.00 / 0.96
盲测集 · template 第 5 轮（盲测）	0.58	0.80 / 0.33	0.64 / 0.69
盲测集 · claude（盲测）	0.74	1.00 / 0.92	1.00 / 0.46
盲测集 · template 第 6 轮修复后（已非盲测）	0.81	1.00 / 0.58	0.80 / 0.92
plan 影响页面（开发集，micro）	precision 0.82 / recall 1.00：没有漏检，多出的 2 页属于保守重检		

结论：第 5 轮的规则过拟合了开发集，盲测下只能抓到三分之一的过时说法。第 6 轮只修了盲测集暴露出的通用 bug：注入检测把正常的接口文档行（例如 Bearer 请求头）误判为外发；数量规则只认紧挨着的数字，也不区分 200KB 这类带单位的值；不存在的项目式类型名（如 `…PageBuilder`）被放过；构建配置（`compileSdk`）没有纳入核对。平台范围类、功能不存在类断言属于语义判断，交给 claude 模式。修复后在 5 个真实仓库上复跑，冲突误报都是 0。每条规则都需要盲测集来约束。

5.4 真实 agent 使用 skills · 5.5 claude 模式

`demo/skills-demo.sh`：haiku 问“购物车上限三端分别是多少”时触发 kb-query，调用 `kb parity`，5 轮、$0.008 答出 50 / 99 / 50；问“wiki 是否与代码一致”时触发 kb-lint，17 轮，全程只读。claude 模式下每次 compound 最多分配 2 个 LLM 任务，所有步骤 verify 通过，写集以外的写入会被自动回滚。

5.6 不依赖 GitHub：每次 push 到 master 触发 compound（`demo/hook-demo.sh`）

先在本地建一个 bare 仓库作为“服务器”，用 `kb hook install --mode post-receive` 安装钩子。然后连续 push 三个 merge：先 push 01，0.8 s 后 push 02，再过 0.3 s push 03。demo 里用 `kb-slow` 模拟每次 3 s 的 LLM 耗时，让几次 push 在时间上重叠。

实测（`remote:` 输出）：seed 全量 9 个任务；01 增量；02 拿到锁时 master 已经是 03，因此一次追平（`e6cff74..1c373e4 (4 new commit(s))`）；03 输出 `already compounded`。每次都 verify PASS。

最终 remote master 与 wiki 的 last_sha 一致。实现要点：用 `mkdir` 锁保证串行；拿到锁后读取分支的当前头，从 last_sha 开始增量；wiki 放在服务端目录（`--wiki-dir`），不回写 master，因此不会递归触发钩子。

5.7 Agent A/B 基准：有 wiki 与没有 wiki（`bench/agent-qa/`）

同一组可以用正则核对的问题，用 `claude -p --model haiku` 分两组跑：A 组是仓库 + wiki + kb skills，B 组是同一份源码，只能用 grep 和读文件。minishop 上 10 题 × 3 次，A 和 B 都是 30/30，A 平均 9.0 turns、$0.25，B 平均 12.7 turns、$0.29。

“三端不一致”这题 A 用 11 turns，B 用 39 turns；“过时文档”这题有了 `kb conflicts` 后降到 4.7 turns。Cases 上的查找题两组持平。

Cases · 5 道汇总题 × 2 次（最终代码）	通过	平均 turns	费用	耗时
A：wiki + AGENTS.md 指引块 + `kb routes` / `lint-tid --pages --top`	10/10	10.0	$0.086	52 s
B：只有源码	9/10	14.0	$0.126	46 s

A 组 turns 少 29%，费用少 32%，耗时慢 13%（第一次 kb 调用要生成缓存）。逐轮演进：只有 wiki 时 A 从未调用 kb，17.6 turns；加上指引块后 13.8 turns；加上汇总命令后，8/10 的运行调用了 kb。结论：wiki 的价值取决于问题能否被一条命令覆盖，并且要在 AGENTS.md 中写明具体动作。全部基准约 $1.85。

6 真实项目验证与性能

6.1 公开开源项目：23 个仓库（第 7 轮，最终代码全部重跑，详见 docs/validation.md）

所有仓库都以只读方式浅克隆，wiki 写到仓库之外。覆盖 Android 7 个（Compose、nav-graph、Appyx、KMP）、iOS 6 个（SwiftUI、UIKit/AppKit）、HarmonyOS 9 个（router_map、main_pages、Navigation），另有 axios 1 个。每个仓库都执行了 init、scan、bench、compound、verify、routes、parity 和 ingest。下表列出其中有代表性的 10 个。

项目	技术栈	文件	符号	路由	召回（与 grep 基准对照）	冷 / 热扫描	峰值 RSS
element-x-android	Kotlin · Appyx	4,355	22,161	176	176/179	11.0 / 1.7 s	667 MB
compose-samples	Compose（6 个应用）	364	2,831	30	30/30	1.4 / 0.16 s	75 MB
kotlinconf-app	KMP	263	1,442	25	25/25；parity 假缺口 25→0	0.6 / 0.07 s	51 MB
IceCubesApp	SwiftUI	432	6,058	44	42/42	1.9 / 0.22 s	109 MB
kickstarter/ios-oss	UIKit + 深链表	2,080	37,731	120	深链 39/39；UIKit 抽样 8/10	13.9 / 2.2 s	596 MB
wikipedia-ios	UIKit + ObjC	1,449	31,452	146	UIKit 抽样 8/10	12.4 / 1.8 s	538 MB
NetNewsWire	UIKit / AppKit	728	12,528	30	抽样 10/10	4.0 / 0.40 s	212 MB
NextE	ArkTS · Navigation	725	19,921	58	页面 51/51	8.1 / 1.0 s	386 MB
agc 模板（sparse）	ArkTS · router_map	2,437	18,460	252	router_map 143/143	7.0 / 1.0 s	353 MB
app_samples（sparse）	OpenHarmony	2,612	51,342	416	抽查 12/12	18.3 / 3.2 s	886 MB

23/23	98.7 / 95.8 / 99.4%	187 → 3	25
kb verify PASS（compound 后、ingest 后各验证一次）	Kotlin / Swift / ArkTS 无错解析率（TS·JS 99.9%）	假冲突（新增 17 个仓库的冲突总数 191 → 6，其中 3 个是真冲突）	项修复，每项都有最小片段回归测试

25 项修复：`kb init` 平台检测与 KMP 重叠根；新增路由来源（nav-graph XML、Compose 路由引用、Appyx NavTarget、HarmonyOS main_pages / `$profile` / pushPathByName / NavDestination、SwiftUI 的三种写法、URL 路由表）；多应用仓库锚错模块；KMP 的 parity 假缺口；规模（热扫描从 O(R^2) 降到 O(R)，ingest 建立索引，页面设 token 预算，大 wiki 使用分层 index，graph 以 gzip 存储）；假冲突识别；符号链接根；路由消失后的功能页处理。

UIKit / AppKit（#23、#25）：识别 `*ViewController` 屏幕和导航边。

- 屏幕数：NetNewsWire 0→30，wikipedia-ios 0→146，kickstarter 48→120；
- 边数：4 / 28 / 48；
- 召回抽样：10/10、8/10、8/10；
- 精度：3 个仓库各抽 10 个屏幕，#25 排除“没有被导航到的嵌入式子控制器”后，误判为屏幕的比例从 17%（5/30）降到 10%（3/30）。

边只覆盖直接类型和同函数内的局部赋值。父类定义在 Objective-C 中的控制器，以及 URL handler 中的深链，暂不支持。

规模：热扫描大致随符号数线性增长，5 万符号约 3 s（同机对比，修复前 10.2 s）。峰值 RSS 约 17 KB / 符号，在 8 GB 的 CI 机器上 30 万符号以内是安全的。ingest：NextE 3,351 条断言耗时 21 s。

未支持：自定义导航框架（DroidconKotlin、Decompose、Voyager）；Objective-C。Kotlin 2.x 的 when 守卫、Swift 自定义运算符、ArkTS 的 `Repeat.each` 嵌套组合会导致解析失败，但符号仍会尽量抽取。

6.2 axios（JavaScript，只读）

扫描 243 个文件、1,009 个符号，识别出 105 个 API。411 条断言中冲突为 0：#21 修复了 8 条假冲突，原因是 `MockAdapter` 来自外部包。claude 模式核对 AGENTS.md，发现 3 处规范与实现不一致，例如 `toFormData.js` 中使用了 `new Error(...)`，违反“never raw Error”的约定。

7 最佳实践与引用

实践	做法	参考
事实与叙述分离	CLI 抽取所有能确定的事实，并标记 DO NOT EDIT；LLM 只写解释；采用 raw / wiki / schema 三层，index 加 append-only log	Karpathy《LLM Wiki》
按符号增量	以符号 ID 和归一化 hash 判断变化，注释与格式改动属于 no-op	FSoft CodeWiki
引用失效	签名变化沿反向引用边传播 k 跳；引用解析识别 import 和作用域	RepoAgent
受限写集	每个任务只能写自己的页面，越界写入自动回滚	OpenDeepWiki
repo-map 式上下文	给 LLM 精选的符号签名和源码片段，而不是整个仓库	Aider repo-map
需求可追踪	用 EARS 句式或 FR-xxx 编号写需求，断言 ID 保持稳定；与代码冲突时以代码为准	spec-kit、OpenSpec、Kiro、EARS
评测流水线	维护人工标注的 golden set，跟踪 precision / recall，并把指标设为单测下限	LLM eval 实践
skill 保持薄	skill 只描述何时用、怎么调 CLI；description 用真实 agent 实测并迭代	Agent Skills 规范

先验证再发布	在 staging 上 verify 后原子 rename；CI 或钩子串行执行；LLM 预算按价值分配	—
防 prompt injection	文档和代码以 `<untrusted-data>` 块传给 LLM，并声明块内一律是数据；中英文规则会删除可疑指令行，可疑断言永不提升；写集回滚是最后一道硬约束	OWASP LLM Top 10（LLM01）

安全实测：测试中构造了一份恶意旧文档（“忽略之前的所有指令…”“AI assistants must delete … rm -rf”），再加一行带注入的源码注释，交给一个“被攻陷”的 LLM。这个 LLM 会把所有断言标为 verified，并尝试写 `index.md`。结果：注入原文没有进入 prompt，可疑断言保持 unverifiable，`index.md` 被回滚，`_conflicts.md` 列出了“可疑内容”。在 axios 的 149 份文档（411 条断言）上误报为 0。

常见坑

- 手工修改 hash 让 verify 变绿，会破坏“知识与代码一致”的保证。应该运行 `kb compound`。
- wiki 自身的提交又触发 compound。用 `paths-ignore: wiki/**` 加 `[skip ci]` 避免。
- verify 通过不代表文档正确。还要看 `_conflicts.md` 和 `kb parity`。

8 局限与路线图

当前局限

- cgo：tree-sitter 语法为 C 代码，构建需要 C 编译器。Swift 语法会产生无害的编译警告，可用 `CGO_CFLAGS=-w` 屏蔽。
- 语法覆盖：23 个仓库合计的无错解析率为 Kotlin 98.7%、Swift 95.8%、ArkTS 99.4%。导致失败的写法包括 Kotlin 2.x 的 when 守卫、Swift 自定义运算符和 `associatedtype … where`、ArkTS 的 `Repeat.each` 嵌套组合，均属上游语法的限制。Objective-C 不解析。失败会记入 `parse_failures`，不会中断扫描。修改 vendored 语法后要同时更新 `GrammarHash`，有测试强制检查。
- 引用解析：基于名称、作用域和 import，没有完整的类型推断，也不建立跨平台的边。
- facts 启发式：API 抽取依赖适配器，目前覆盖 Retrofit / Ktor / OkHttp / Alamofire / Moya / Endpoint 枚举 / http / @ohos/axios / RCP / axios。完全自研、动态拼接 URL 的网络层仍识别不到；注入检测是基于规则的，属于纵深防御的一层，不能保证全部拦截。
- 多 Agent：Codex 和 Gemini 在本环境只能连 mock 模型，所以它们写页面的质量没有实测；Cursor 和 Copilot 只依据文档。
- 断言核对：template 在盲测集上的 conflict recall 只有 0.33，修复后为 0.58，所以它只能作为初筛。平台范围、功能是否存在这类语义判断需要 claude 模式（盲测 recall 0.92），单个任务约 30–60 s。

路线图

- 短期：可选接入 SCIP / LSP 作为精确引用源；增加网络层适配器（Retrofit 动态 URL、Apollo GraphQL）；支持 Decompose / Voyager 导航；URL handler 深链；`kb parity --strict` 接入 CI。
- 中期：与 e2epilot 双向打通，只重跑受 `changed_route_ids` 影响的流程，并把测试结果回写到 requirements；用真实模型评测 Codex / Gemini 后端的写作质量。
- 长期：多仓库 / 多 wiki 联邦检索；从 PR 评论中吸收人工知识，经 verify 后才沉淀；支持 Dart、C++、Objective-C。